Quack Stack TM

Today's Agenda...

01 Introduction

02 Phylosophy Of Juan Pato

03 Meaningful Atoms

04 Friends Of Order

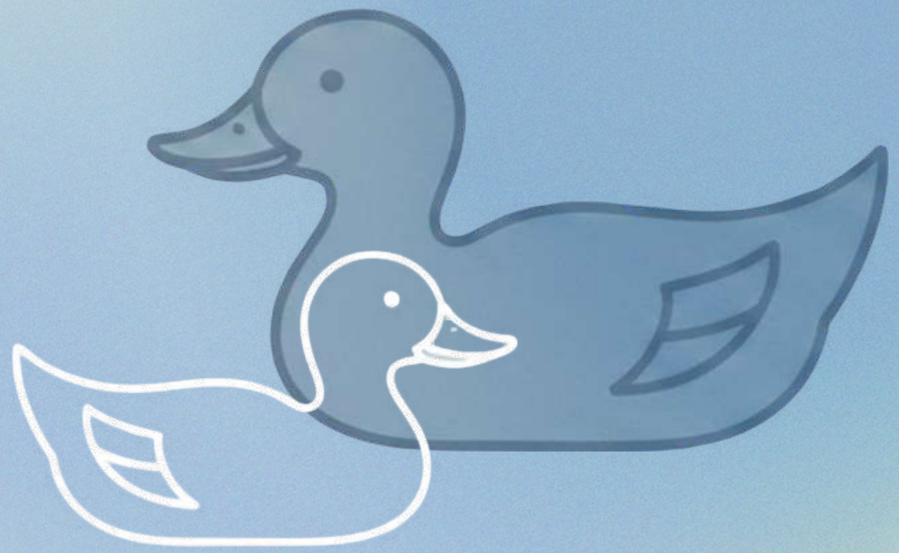
05 Not A Transator, A Gatekeeper

06 The Road to innovation

Presented by :

Quack Stack TM

UNAM ENGINEERING FACULTY



Juan pato v0.1

The architecture of meaning

AMBIGUITY

GRAMMAR (G)

E → E + E | E * E | (E) | id
S → if E thenS else S | if E thenS
S → while E do S | { L }
L → L ; S | S

```
if a < b then
   if c < d then
      x = y + z * w
   else
      x = y * z + w
```

Which 'if' does else match?

PARSER TABLE (LR(1))

State	id	+	(	)	$	Action
5		S5				...
6		r2				r2 / s7
						r4 / s8
						...

Syntax Error
Unexpected token 'else' in input
OK

Syntax Error
Unexpected token '*'
Expected ')'
OK

```
int func(int a) {
  if (a < 10)
     return a * a + 1;
  else
     return a * (a + 1);
```

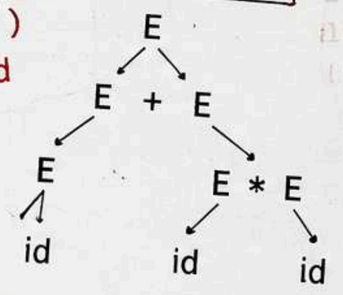
Syntax Error
Reduce/Reduce conflict in state 12
OK

```
x = a + b * c + d
```

(a + b) * (c + d) ?
a + (b * (c + d)) ?
(a + (b * c)) + d ?

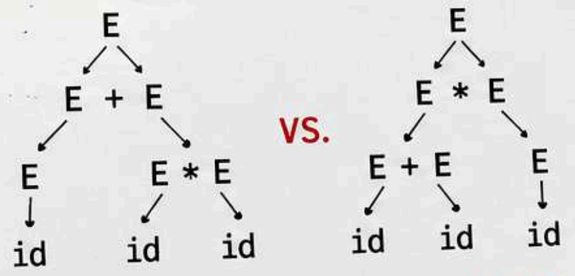
E → E + E
E → E * F
E → (E)
E → id

Ambiguous Grammar



Multiple Parse Trees!

ALL POSSIBLE PARSE TREES



Same input, different trees, different meanings!

```
while i < n do
  if a[i] == 0 then
     i = i + 1
  else
     break
```

Dangling else problem!

Syntax Error
Ambiguous grammar: multiple parses possible
OK

AMBIGUITY
Leads to:
- Shift/Reduce Conflicts
- Reduce/Reduce Conflicts
- Uncertain Parse Trees
- Wrong Meaning!

Quack Stack TM

Phylosophy of Juan Pato

“Syntactic precision, visionary execution”

- Leonardo Leyva, CEO

Quack Stack TM

A language designed for clarity and safety

```
ABSTRACT SYNTAX TREE:
←[93m
              program

      _________|_____

   write              int

     |            _____|___

"This is an     a         10

  example"
```

```
-----------------------------------
Starting semantic analysis...
-----------------------------------
←[0m
←[92mSDT Verified!←[0m
```

```
Total tokens identified: 13

Lexical analysis completed.
```

```
Starting syntactic analyzer...
-----------------------------------
←[0m
←[92m
Parsing Success!←[0m
```

PIPELINE

LEXICAL

Breaking code into tokens

Converts a stream of characters into meaningful tokens.

if, (, id, <, num,), {, ...

→

SYNTACTIC

Checking structure

Verifies that the token sequence follows the grammar rules.

Parse Tree / AST

→

SEMANTIC

Understanding meaning

Analyzes meaning, types, and context to ensure the program makes sense.

Symbol Table / Type Info / Checks

Quack Stack TM

Meaningful Atoms

How does the compiler transform a sequence of characters into tokens?

INTEGER = (– | ε) D +

```
Tokens Identified:

TOKEN TYPE          VALUE
----------------------------------------
START               Start
L_BRACKET           {
WRITING_OP          printf
L_PARENTHESYS       (
STRING              "This is an example"
R_PARENTHESYS       )
SEMI_COLON          ;
TYPE                int
ID                  a
ASIG_OP             =
INTEGER             10
SEMI_COLON          ;
R_BRACKET           }
----------------------------------------
```

Friends Of Order

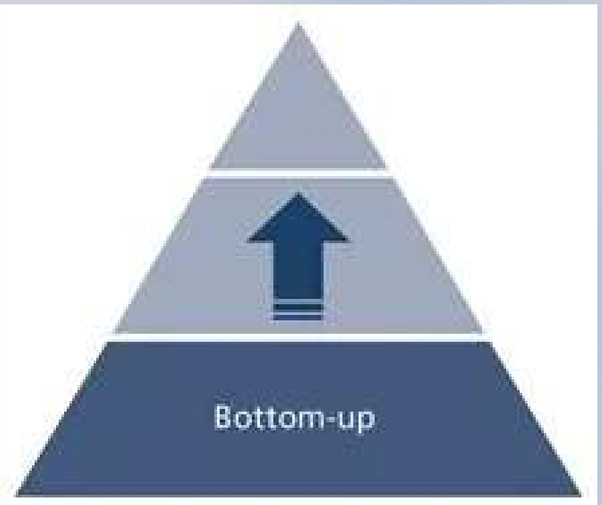
PARSER: WHAT IS IT?

Converts raw tokens into an organized hierarchy, a structure called Abstract Syntax Tree (AST).

```
ABSTRACT SYNTAX TREE:
←[93m
            program

     _________|____

   write            int

     |          ____|__

"This is an     a        10

  example"
```

BOTTOM UP

Provides a greater flexibility by building the logic, from specific details (leaves), to the general structure (root).




```
precedence=[
    ('left', ['BOOL_OP']),
    ('left', ['COMP_OP']),
    ('left', ['ARIT_OP']),
])
```

PRECEDENCE ORDER

Resolves ambiguity and ensures the parser builds the logical hierarchy required for correct evaluations.

Our Grammar is...

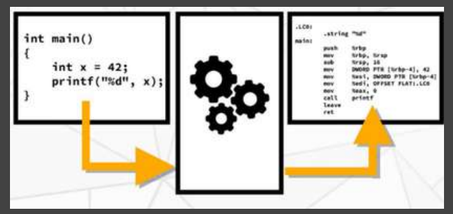
A Context Free Grammar

```
@self.pg.production('program : START L_BRACKET instructions R_BRACKET')
@self.pg.production('instructions : instruction instructions')
@self.pg.production('instructions : instruction')
```

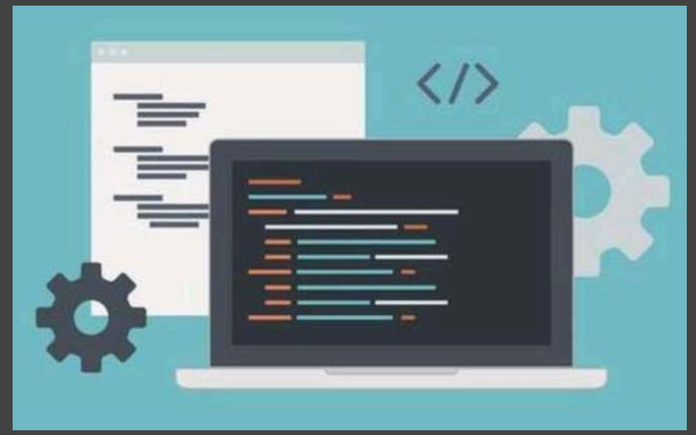
And Recursive

```
@self.pg.production('instructions : instruction instructions')
@self.pg.production('expression : expression ARIT_OP expression')
@self.pg.production('expression : expression COMP_OP expression')
@self.pg.production('expression : expression BOOL_OP expression')
```

Quack Stack TM



Not a Translator, a Gatekeeper



PERFECT LOGIC GOES INTO PRODUCTION

TWO TOOLS

- THE TREE

- THE SYMBOL TABLE

ERRORS

- SYNTACTIC
- SEMANTIC

The Road to innovation

Version 1 — **Adoption** — **Version 2** — **Expansion**

Version 1 (Academic Edition)

- Free access for universities
- Focused on learning and experimentation
- Easy to use and understand

Adoption Phase

- Students and institutions start using it
- Familiarity and trust are built
- Becomes part of the learning process

Version 2 (Professional Edition)

- Paid solution
- Modular design
- Adaptable to real-world needs

Expansion

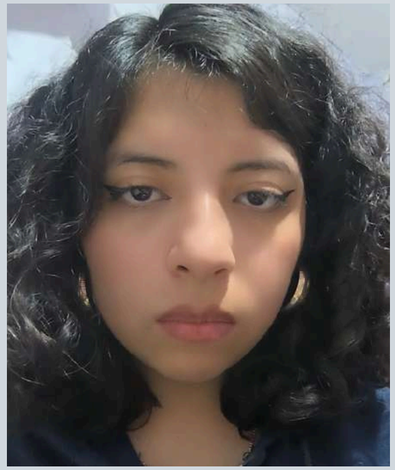
- Industry use
- Continuous updates
- Professional support and maintenance

Quack Stack TM

Meet Our Team

KARINA ROSETE
LEAD ENGINEER

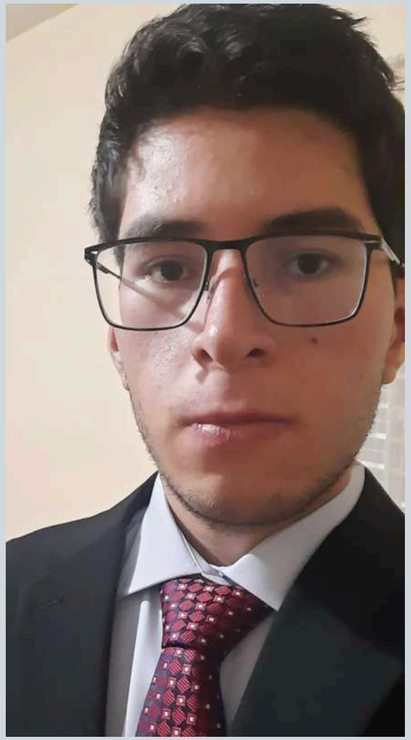
NILSEN FLORES
ENGINEER



SARA LOPEZ
ENGINEER

Quack Stack TM

Meet Our Team



LEONARDO LEYVA

CEO

VICTOR

PROJECT MANAGER

Presented by :
Quack Stack Team

Thank You

for your time and attention